1.3 개발 환경 설정 가이드 (Windows)

개요

Polkadot SDK를 사용한 블록체인 개발을 시작하기 위해서는 적절한 개발 환경 설정이 필요합니다. 이 가이드는 Windows 환경에서 Polkadot SDK 개발에 필요한 모든 도구와 의존성을 설치하고 설정하는 과정을 단계별로 설명합니다.

시스템 요구사항

최소 요구사항

- **운영체제**: Windows 10 (64-bit) 이상 또는 Windows 11
- **메모리**: 8GB RAM (16GB 권장)
- **저장공간**: 50GB 이상의 여유 공간
- **프로세서**: Intel i5 또는 AMD Ryzen 5 이상

권장 사양

- **메모리**: 32GB RAM
- **저장공간**: SSD 100GB 이상
- **프로세서**: Intel i7/i9 또는 AMD Ryzen 7/9
- **네트워크**: 안정적인 인터넷 연결

1단계: 기본 도구 설치

1.1 Windows Subsystem for Linux (WSL) 설치 (권장)

WSL을 사용하면 Windows에서 Linux 환경을 실행할 수 있어 개발이 더욱 편리합니다.

```
# PowerShell을 관리자 권한으로 실행
wsl --install


# 재부팅 후 Ubuntu 설치 확인
wsl --list --verbose


# Ubuntu 업데이트
wsl -d Ubuntu
sudo apt update && sudo apt upgrade -y
```

1.2 Git 설치

```
# Windows용 Git 다운로드 및 설치
# https://git-scm.com/download/win 에서 다운로드


# 설치 확인
git --version
```

1.3 Visual Studio Code 설치

```
# VS Code 다운로드 및 설치
# https://code.visualstudio.com/ 에서 다운로드

# 필수 확장 프로그램 설치
# - rust-analyzer
# - WSL (WSL 사용 시)
# - GitLens
# - Better TOML
```

2단계: Rust 개발 환경 설정

2.1 Rust 설치

Windows 네이티브 설치

```
# Rustup 다운로드 및 설치
# https://rustup.rs/ 에서 rustup-init.exe 다운로드 후 실행

# 또는 PowerShell에서 직접 설치
Invoke-WebRequest -Uri "https://win.rustup.rs/" -OutFile "rustup-init.exe"
.\rustup-init.exe

# 설치 확인
rustc --version
```

```
cargo --version
```

WSL 환경에서 설치

```
# WSL Ubuntu 환경에서 실행
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# 환경 변수 적용
source ~/.cargo/env

# 설치 확인
rustc --version
cargo --version
```

2.2 Rust 도구체인 설정

```
# 안정 버전 설치 및 기본 설정
rustup default stable

# WebAssembly 타겟 추가 (Substrate 런타임 컴파일용)
rustup target add wasm32-unknown-unknown

# 나이틀리 버전 설치 (일부 기능용)
```

```
rustup install nightly
rustup target add wasm32-unknown-unknown --toolchain nightly


# 유용한 Rust 도구 설치
cargo install cargo-edit
cargo install cargo-expand
cargo install cargo-watch
```

2.3 추가 개발 도구 설치

```
# Substrate 개발에 유용한 도구들
cargo install --git https://github.com/paritytech/substrate subkey
cargo install --git https://github.com/paritytech/polkadot-sdk try-runtime-c
cargo install cargo-contract
```

3단계: 시스템 의존성 설치

3.1 Windows 네이티브 환경

```
# Visual Studio Build Tools 설치 (C++ 컴파일러)
# https://visualstudio.microsoft.com/visual-cpp-build-tools/ 에서 다운로드


# 또는 Chocolatey 사용
```

```
Set-ExecutionPolicy Bypass -Scope Process -Force

[System.Net.ServicePointManager]::SecurityProtocol = [System.Net.ServicePointMan

iex ((New-Object System.Net.WebClient).DownloadString('https://community.chocola


# 필수 도구 설치

choco install visualstudio2022buildtools

choco install llvm

choco install cmake

choco install protoc
```

3.2 WSL 환경

```
# 필수 패키지 설치

sudo apt update

sudo apt install -y \

build-essential \

clang \

curl \

git \

make \

libssl-dev \

llvm \

libudev-dev \
```

```
protobuf-compiler \
pkg-config

# 추가 개발 도구
sudo apt install -y \
cmake \
python3-pip \
nodejs \
npm
```

4단계: Polkadot SDK 설치 및 빌드

4.1 소스 코드 다운로드

```
# Polkadot SDK 저장소 클론
git clone https://github.com/paritytech/polkadot-sdk.git
cd polkadot-sdk

# 최신 안정 버전으로 체크아웃
git checkout $(git describe --tags --abbrev=0)
```

4.2 빌드 및 테스트

```
# 전체 프로젝트 빌드 (시간이 오래 걸림)
cargo build --release

# 특정 바이너리만 빌드
cargo build --release -p polkadot
cargo build --release -p polkadot-parachain-bin

# 빌드 확인
./target/release/polkadot --version
./target/release/polkadot-parachain --version

# 기본 테스트 실행
cargo test --workspace --release
```

4.3 빠른 시작 스크립트 실행

```
# 자동 설정 스크립트 실행
curl --proto '=https' --tlsv1.2 -sSf https://raw.githubusercontent.com/pari
```

5단계: 개발 도구 및 IDE 설정

5.1 Visual Studio Code 설정

필수 확장 프로그램

```
// .vscode/extensions.json
{
"recommendations": [
"rust-lang.rust-analyzer",
"ms-vscode-remote.remote-wsl",
"eamodio.gitlens",
"tamasfe.even-better-toml",
"vadimcn.vscode-lldb",
"serayuzgur.crates",
"dustypomerleau.rust-syntax"
]
}
```

작업 공간 설정

```
// .vscode/settings.json
{
"rust-analyzer.cargo.features": "all",
"rust-analyzer.checkOnSave.command": "clippy",
"rust-analyzer.checkOnSave.allTargets": false,
"rust-analyzer.cargo.buildScripts.enable": true,
"rust-analyzer.procMacro.enable": true,
```

```
"files.watcherExclude": {
"**/target/**": true
}
}
```

디버깅 설정

```
// .vscode/launch.json
{
"version": "0.2.0",
"configurations": [
{
"type": "lldb",
"request": "launch",
"name": "Debug Substrate Node",
"cargo": {
"args": [
"build",
"--bin=polkadot",
"--package=polkadot"
],
"filter": {
"name": "polkadot",
```

```
"kind": "bin"
}
},
"args": ["--dev"],
"cwd": "${workspaceFolder}"
}
]
}
```

5.2 개발 환경 최적화

Cargo 설정 최적화

```
# ~/.cargo/config.toml
[build]
rustc-wrapper = "sccache" # 컴파일 캐싱 (선택사항)

[target.x86_64-pc-windows-msvc]
linker = "rust-lld"

[net]
git-fetch-with-cli = true

[registries.crates-io]
```

```
protocol = "sparse"
```

환경 변수 설정

```
# Windows 환경 변수 설정
[Environment]::SetEnvironmentVariable("RUST_LOG", "info", "User")
[Environment]::SetEnvironmentVariable("WASM_BUILD_TOOLCHAIN", "nightly", "User")
```

```
# WSL 환경 변수 설정 (~/.bashrc에 추가)
export RUST_LOG=info
export WASM_BUILD_TOOLCHAIN=nightly
export CARGO_INCREMENTAL=0
export RUSTFLAGS="-D warnings"
```

6단계: 개발 네트워크 도구 설치

6.1 Zombienet 설치

```
# Zombienet 바이너리 다운로드 (Windows)
curl -LO https://github.com/paritytech/zombienet/releases/latest/download/zomb
mv zombienet-windows.exe zombienet.exe

# 실행 권한 부여 및 PATH 추가
```

```
chmod +x zombienet.exe
```

6.2 Polkadot.js Apps 로컬 실행

```
# Node.js 및 npm 설치 확인
node --version
npm --version

# Polkadot.js Apps 클론 및 실행
git clone https://github.com/polkadot-js/apps.git
cd apps
npm install
npm start
```

6.3 Docker 설정 (선택사항)

```
# Docker Desktop 설치
# https://www.docker.com/products/docker-desktop 에서 다운로드

# WSL 2 백엔드 활성화
# Docker Desktop 설정에서 "Use the WSL 2 based engine" 체크

# 설치 확인
```

```
docker --version

docker-compose --version
```

7단계: 첫 번째 노드 실행

7.1 개발 노드 실행

```
# Polkadot 개발 노드 실행

./target/release/polkadot  --dev  --tmp


# 또는 Substrate 노드 템플릿 실행

git  clone  https://github.com/substrate-developer-hub/substrate-node-template.g

cd  substrate-node-template

cargo  build  --release

./target/release/node-template  --dev
```

7.2 연결 확인

1. 웹 브라우저에서 [Polkadot.js Apps](#) 접속
2. 좌측 상단에서 “Local Node” 선택
3. 연결 상태 확인 및 블록 생성 모니터링

8단계: 성능 최적화 및 문제 해결

8.1 컴파일 성능 최적화

```
# 병렬 컴파일 작업 수 설정
export CARGO_BUILD_JOBS=4

# 링커 최적화 (Windows)
export RUSTFLAGS="-C link-arg=-fuse-ld=lld"

# 메모리 사용량 제한
export CARGO_BUILD_RUSTC_WRAPPER_RELATIVE_PID=1
```

8.2 일반적인 문제 해결

컴파일 오류

```
# 캐시 정리
cargo clean

# 의존성 업데이트
cargo update

# 특정 크레이트 재빌드
cargo build --release -p polkadot --force-rebuild
```

메모리 부족 오류

```
# 스왑 파일 크기 증가 (WSL)
sudo fallocate -l 8G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
```

네트워크 연결 문제

```
# 방화벽 설정 확인
# Windows Defender 방화벽에서 포트 30333, 9944, 9933 허용

# WSL 포트 포워딩 설정
netsh interface portproxy add v4tov4 listenport=9944 listenaddress=0.0.0.0
```

9단계: 추가 도구 및 리소스

9.1 유용한 개발 도구

```
# 코드 품질 도구
cargo install cargo-clippy
cargo install cargo-fmt
cargo install cargo-audit
```

```
# 벤치마킹 도구
cargo install cargo-criterion

# 문서 생성 도구
cargo install mdbook
```

9.2 모니터링 도구

```
# 시스템 모니터링
cargo install bottom # htop 대안
cargo install bandwhich # 네트워크 모니터링

# 로그 분석
cargo install lnav
```

9.3 개발 리소스

- **공식 문서**: docs.polkadot.com
- **Rust 문서**: paritytech.github.io/polkadot-sdk
- **커뮤니티**:
- Substrate Stack Exchange
- Polkadot Discord
- Telegram

환경 검증 체크리스트

설치가 완료되면 다음 명령어들이 정상적으로 작동하는지 확인하세요:

```
# ✅ 기본 도구 확인
git --version
rustc --version
cargo --version

# ✅ Rust 타겟 확인
rustup target list --installed | grep wasm32-unknown-unknown

# ✅ Polkadot SDK 빌드 확인
cd polkadot-sdk
cargo check --workspace

# ✅ 노드 실행 확인
./target/release/polkadot --version

# ✅ 개발 노드 실행 테스트
timeout 30s ./target/release/polkadot --dev --tmp
```

결론

이 가이드를 따라 설정을 완료하면 Windows 환경에서 Polkadot SDK를 사용한 블록체인 개발을 시작할 수 있습니다. 개발 환경 설정은 한 번만 수행하면 되지만, 정기적인 업데이트와 최적화를 통해 더 나은 개발 경험을 얻을 수 있습니다.

다음 단계로는 Substrate 기초 튜토리얼을 진행하거나, 파라체인 템플릿을 사용해 첫 번째 프로젝트를 시작해보는 것을 권장합니다.